ADEKUNLE AJASIN UNIVERSITY, AKUNGBA-AKOKO
DEPARTMENT OF COMPUTER SCIENCE

CSC426 – WEEKLY DEV, WEEKEND 3 (01/07/26)

# IRO – ISSUES REPORTING OUTLET

A web-based system for relating and reporting issues concerning
AAUA students and indigenes of Akungba, Iwaro, Etioro, Ayegunle and environs



| | |
|---|---|
| **Name** | Joshua Omolafe |
| **Matric No.** | |
| **Course lecturer** | Dr. O. O. Ajayi |
| **GitHub repository** | github.com/Joshua989/Weekend-3-01-07-26- |



## Contents



| Document | Page |
|---|---|
| 1. Software Requirements Specification (SRS) | 2–13 |
| 2. Use-Case Diagram | 14 |
| 3. Ishikawa (Fishbone) Problem Analysis | 15 |
| 4. Object Listing | 16–18 |
| 5. Code Listing | 19–34 |

# SOFTWARE REQUIREMENTS SPECIFICATION

## IRO – Issues Reporting Outlet

A web-based system for reporting and resolving issues between
Adekunle Ajasin University, Akungba-Akoko (AAUA) students and the indigenes of
Akungba, Iwaro, Etioro, Ayegunle and neighbouring communities

Version 1.0

CSC426 Weekly Dev – Weekend 3 (01/07/26)
Course lecturer: Dr. O. O. Ajayi

Prepared by: Joshua Omolafe (GitHub: Joshua989)

### Revision History

| Version | Date | Description |
|---|---|---|
| 1.0 | July 2026 | Initial SRS for the IRO mini web interface. |

### Table of Contents

1. Introduction
2. Overall Description
3. System Architecture & Interaction of Major Components
4. Functional Requirements
5. Non-Functional Requirements
6. External Interface Requirements
7. Use-Case Model
8. Data Requirements
9. Problem Analysis (Ishikawa) and Requirements Traceability
10. Appendices

## 1. Introduction

### 1.1 Purpose

This document specifies the requirements of **IRO (Issues Reporting Outlet)**, a web-based system through which AAUA students and indigenes of the host communities report issues that affect them, and through which an IRO desk officer reviews, escalates and resolves those reports. It is written for the course lecturer, the project developer, and any future team that extends the system. It describes what the system must do (functional requirements), how well it must do it (non-functional requirements), and how its major components interact.

### 1.2 Scope

Relations between AAUA students and the host communities (Akungba, Iwaro, Etioro, Ayegunle and Oke-Oka) are affected by disputes over rent and accommodation, security, land, cultural festivals, sanitation, transport and prices. Today these issues are reported through scattered informal channels (the SUG, community chiefs, WhatsApp groups, the police), many are never reported for fear of victimisation, and reporters rarely learn what happened to their complaint.

IRO provides a single, neutral outlet that:

- lets any student or indigene submit an issue report, anonymously if they wish;
- gives every report a unique ticket ID that the reporter uses to track its progress;
- gives the IRO desk officer a dashboard to view, filter, update, escalate and close reports;
- keeps a timestamped history of every status change for accountability.

**Out of scope for version 1.0 (the mini interface):** SMS/e-mail notifications, file/photo uploads, a server-side database, multiple officer accounts and Yoruba translation. These are listed as future enhancements in Appendix B.

### 1.3 Definitions, Acronyms and Abbreviations

| Term | Meaning |
|---|---|
| IRO | Issues Reporting Outlet – the system specified here. |
| AAUA | Adekunle Ajasin University, Akungba-Akoko, Ondo State, Nigeria. |
| Indigene | A native or long-term resident of Akungba, Iwaro, Etioro, Ayegunle, Oke-Oka or neighbouring communities. |
| Reporter | Any user (student, indigene or other) who submits a report. |
| Desk Officer | The administrator who manages reports on the IRO desk. |
| Escalation Authority | A body to which a report is forwarded: Students' Affairs, SUG, community leaders (Oba/Baale), campus security or police. |
| Ticket ID | Unique report reference in the form `IRO-YY-XXXXX`, e.g. IRO-26-4F7K2. |
| SRS | Software Requirements Specification. |
| FR / NFR | Functional requirement / non-functional requirement. |
| UI | User interface. |

### 1.4 References

- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- IRO Community Needs Questionnaire (Google Form) – `docs/questionnaire/`.
- IRO Ishikawa Problem Analysis – `docs/ishikawa/IRO_Ishikawa.drawio`.

- IRO Use-Case Diagram – `docs/use-case/IRO_UseCase.drawio`.
- Nigeria Data Protection Act (NDPA) 2023.

### 1.5 Overview

Section 2 describes the product in general terms. Section 3 shows the architecture and how the major components interact. Sections 4 and 5 list the functional and non-functional requirements. Section 6 covers external interfaces, Section 7 the use cases, Section 8 the data, and Section 9 traces the requirements back to the root causes found in the Ishikawa analysis.

## 2. Overall Description

### 2.1 Product Perspective

IRO is a new, self-contained web application. Version 1.0 is a static front-end (HTML, CSS, JavaScript) that runs in any modern browser and stores data in the browser's `localStorage`. All data access goes through a single data-layer object (`IssueStore`), so a later version can replace browser storage with a server API and database without changing the user interface.

### 2.2 Product Functions (summary)

- Present the purpose of IRO, reporting guidelines and issue categories.
- Capture an issue report with reporter type, category, severity, location, date, title and description.
- Allow anonymous reporting.
- Validate input and generate a unique ticket ID.
- Let reporters track status and history with the ticket ID.
- Let the desk officer log in, view statistics, search/filter reports, view details, change status with a note, escalate and delete reports.

### 2.3 User Classes and Characteristics

| User class | Description | Technical skill |
|---|---|---|
| Student | Undergraduate or postgraduate student of AAUA, on or off campus. May supply a matric number. | Medium–high; mostly smartphone users. |
| Indigene | Landlord, trader, driver, farmer, chief or other resident of the host communities. Selects their community. | Low–medium; a simple, clear interface is essential. |
| Other reporter | Staff, visitors, traders not from the communities. | Varies. |
| IRO Desk Officer | Staff member (e.g. from Students' Affairs or a joint town–gown committee) who manages reports. | Medium; trained to use the dashboard. |
| Escalation Authority | Secondary actor that receives escalated issues offline; does not log in to v1.0. | N/A |

### 2.4 Operating Environment

- Client: any modern browser (Chrome, Firefox, Edge, Safari, Opera Mini excluded) on Android, iOS, Windows, macOS or Linux.
- Server: none required for v1.0; the files can be opened locally or hosted on any static host (e.g. GitHub Pages).

### 2.5 Design and Implementation Constraints

- Must be simple to build and submit within one week (course deadline).
- Plain HTML5, CSS3 and JavaScript (ES6); no frameworks or build tools.

- Data in v1.0 is held in the browser, so reports are visible only on the device where they were made.
- Must respect the confidentiality of reporters in line with the NDPA 2023.

### 2.6 Assumptions and Dependencies

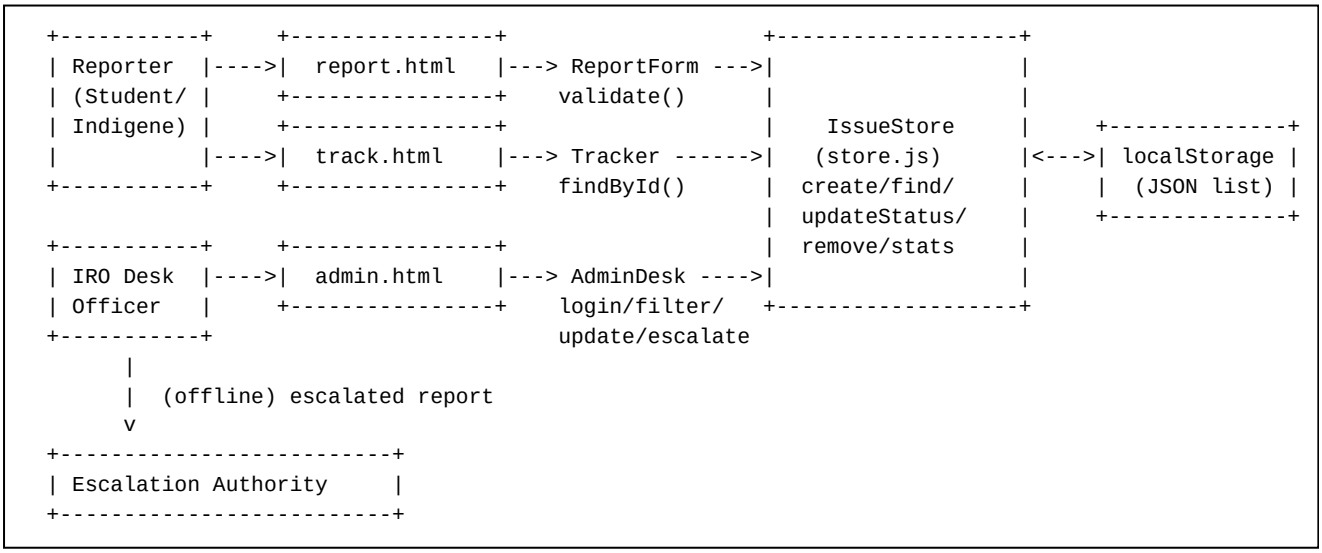
- Users have access to a browser with JavaScript enabled.
- The university and community leaders agree to act on escalated reports.
- A production deployment will add a server-side database and proper authentication.

## 3. System Architecture & Interaction of Major Components

### 3.1 Major Components

| Component | Files | Responsibility |
|---|---|---|
| Presentation layer (pages) | index.html, report.html, track.html, admin.html, css/style.css | Displays the UI: home & guidelines, report form, tracking page and admin desk. |
| ReportForm controller | js/report.js | Shows/hides student or indigene fields, collects and validates input, calls IssueStore.create() and shows the ticket ID. |
| Tracker controller | js/track.js | Looks up a ticket ID via IssueStore.findById() and renders status and history (no personal data shown). |
| AdminDesk controller | js/admin.js | Handles login/logout, statistics, search/filter, detail view, status updates, escalation and deletion. |
| IssueStore (data layer) | js/store.js | Single point of access to data: create, read, find, update status, delete, statistics, ticket ID generation. |
| IRO configuration | js/store.js | Holds categories, communities, statuses, severities and the demo passcode. |
| Storage | Browser localStorage (key `iro_issues`) | Persists the list of issues as JSON. Replaced by a server database in future. |

### 3.2 Component Interaction Diagram



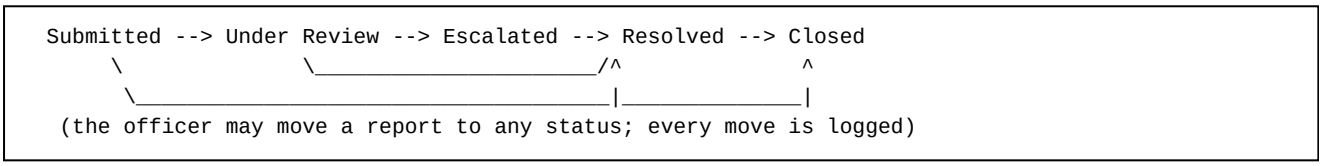
### 3.3 Key Interaction Sequences

**(a) Submitting a report.** Reporter opens report.html → selects reporter type (ReportForm shows matric field for students or community field for indigenes) → fills the form → ReportForm.validate() checks every field → IssueStore.create() builds the issue object, generates a ticket ID, sets status *Submitted*, adds the first history entry and saves to storage → ReportForm displays the ticket ID and a "Track this report" link.

**(b) Tracking a report.** Reporter enters the ticket ID on track.html (or follows the link) → Tracker calls IssueStore.findById() → if found, the title, category, severity, current status and full history are shown; otherwise an error message is shown. Name, contact and description are never shown on this page.

**(c) Managing a report.** Desk officer logs in on admin.html → AdminDesk loads IssueStore.stats() and IssueStore.getAll() → officer searches/filters and opens a report → officer chooses a new status (*Under Review*, *Escalated*, *Resolved*, *Closed*) and writes a note → IssueStore.updateStatus() records the change with a timestamp → the reporter sees the new status and note the next time they track the ticket. Choosing *Escalated* means the officer forwards the case to the relevant authority.

### 3.4 Report Status Life-Cycle




## 4. Functional Requirements

Priority: **H** = must have (in v1.0), **M** = should have, **L** = could have (future).

| ID | Requirement | Description | Pri. |
|---|---|---|---|
| **4.1 Public information** | | | |
| FR-01 | Home page | The system shall display the purpose of IRO, the four-step reporting process, the list of issue categories and an emergency notice. | H |
| FR-02 | Navigation | Every page shall have a navigation bar linking Home, Report an Issue, Track Report and Admin. | H |
| **4.2 Issue reporting** | | | |
| FR-03 | Reporter type | The system shall require the reporter to choose Student, Indigene or Other. | H |
| FR-04 | Conditional fields | If Student is chosen the system shall show an optional matric-number field; if Indigene is chosen it shall show a required community list (Akungba, Iwaro, Etioro, Ayegunle, Oke-Oka, Other). | H |
| FR-05 | Anonymous reporting | The system shall let the reporter tick "Report anonymously"; the name and contact fields shall then be hidden and shall not be stored. | H |
| FR-06 | Issue details | The system shall capture category, severity (Low, Medium, High, Emergency), location, incident date, short title and description. | H |
| FR-07 | Validation | The system shall reject a report and show a clear message if a required field is empty, the date is in the future, the title is shorter than 5 characters, the description is shorter than 20 characters, or a contact given is not a valid phone number or e-mail. | H |

| | | | |
|---|---|---|---|
| FR-08 | Ticket generation | On a valid submission the system shall create a unique ticket ID of the form IRO-YY-XXXXX (no ambiguous characters 0, O, 1, I) and set status to Submitted. | H |
| FR-09 | Confirmation | The system shall display the ticket ID with a link to track the report. | H |
| **4.3 Tracking** | | | |
| FR-10 | Track by ticket | The system shall let anyone enter a ticket ID (case-insensitive) and view the report's title, category, severity, date, current status and status history with notes. | H |
| FR-11 | Privacy on tracking | The tracking page shall not display the reporter's name, contact, matric number or description. | H |
| FR-12 | Unknown ticket | If no report matches, the system shall show "No report found with that ticket ID". | H |
| **4.4 Desk officer (administration)** | | | |
| FR-13 | Login | The admin desk shall be accessible only after a correct passcode is entered; the session ends on logout or when the browser tab is closed. | H |
| FR-14 | Dashboard statistics | The system shall show total reports, open reports, active emergencies and resolved reports. | H |
| FR-15 | List, search & filter | The system shall list all reports newest first and allow search by ticket ID, title or location, and filtering by status, category and reporter type. | H |
| FR-16 | View details | The system shall show a report's full details, including reporter identity (or "Anonymous") and history. | H |
| FR-17 | Update status | The officer shall be able to set a new status with an optional note; the system shall append a timestamped history entry. | H |
| FR-18 | Escalate | The officer shall be able to mark a report as Escalated with a note naming the authority it was forwarded to. | H |
| FR-19 | Delete | The officer shall be able to delete a report after confirming (e.g. spam or duplicates). | M |
| **4.5 Future functions** | | | |
| FR-20 | Notifications | The system should send SMS/e-mail alerts to reporters who give contact details when status changes. | L |
| FR-21 | Evidence upload | The system should allow photo/video attachments. | L |
| FR-22 | Reports & trends | The system should produce monthly summaries by category and community. | L |

## 5. Non-Functional Requirements

| ID | Attribute | Requirement |
|---|---|---|
| **5.1 Performance** | | |
| NFR-01 | Response time | Each page shall load and become usable within 2 seconds on a 3G mobile connection; form submission and search shall respond within 1 second for up to 1,000 stored reports. |
| NFR-02 | Page weight | Total size of the app shall stay under 200 KB, with no external libraries, so it works on low-data plans. |

| | | |
|---|---|---|
| **5.2 Security & Privacy** | | |
| NFR-03 | Confidentiality | Anonymous reports shall store no name or contact. Identifying data shall be visible only on the admin desk. |
| NFR-04 | Access control | Admin functions shall require authentication. (v1.0 uses a demo passcode; production shall use server-side authentication with hashed passwords and HTTPS.) |
| NFR-05 | Input safety | All user text shall be HTML-escaped before display to prevent script injection (XSS). |
| NFR-06 | Ticket secrecy | Ticket IDs shall be random ($32^5 \approx 33$ million combinations per year) so they cannot easily be guessed. |
| NFR-07 | Data protection | Handling of personal data shall comply with the Nigeria Data Protection Act 2023. |
| **5.3 Usability & Accessibility** | | |
| NFR-08 | Ease of use | A first-time user shall be able to submit a report in under 3 minutes without help. |
| NFR-09 | Responsive design | The UI shall work on screens from 320 px (small phones) to desktop without horizontal scrolling. |
| NFR-10 | Accessibility | All inputs shall have visible labels, keyboard focus shall be visible, and text contrast shall meet WCAG 2.1 AA. |
| NFR-11 | Clear language | Messages shall be in plain English; Yoruba support is a future enhancement. |
| **5.4 Reliability & Availability** | | |
| NFR-12 | Data integrity | Every status change shall be logged; history entries shall never be edited or removed except when the whole report is deleted. |
| NFR-13 | Fault tolerance | If stored data is unreadable the system shall start with an empty list rather than crash. |
| NFR-14 | Availability | When hosted, the system shall be available 99% of the time (excluding planned maintenance). |
| **5.5 Maintainability & Portability** | | |
| NFR-15 | Modularity | All data access shall go through IssueStore so storage can be replaced without changing UI code. |
| NFR-16 | Configurability | Categories, communities, statuses and severities shall be defined in one configuration object (IRO). |
| NFR-17 | Portability | The app shall run in current versions of Chrome, Firefox, Edge and Safari with no installation. |
| NFR-18 | Code quality | Source shall be commented and kept in a public GitHub repository with documentation. |
| **5.6 Social / Ethical** | | |
| NFR-19 | Neutrality | The system shall treat students and indigenes equally; the wording of forms shall not blame either group. |

## 6. External Interface Requirements

### 6.1 User Interfaces

| Screen | Main elements |
|---|---|

| | |
|---|---|
| Home (index.html) | Banner, "Report an Issue" and "Track My Report" buttons, 4-step process cards, categories, emergency notice. |
| Report (report.html) | Reporter type, matric/community, anonymous checkbox, name, contact, category, severity, location, date, title, description, Submit; success panel with ticket ID. |
| Track (track.html) | Ticket ID box, Track button, result card with status badge and timeline. |
| Admin (admin.html) | Login form; dashboard with 4 statistic cards, search and filters, report table, detail panel with history, status update form and Delete button. |

### 6.2 Hardware Interfaces

None beyond a device capable of running a modern web browser.

### 6.3 Software Interfaces

Browser Web Storage API (localStorage, sessionStorage). Future: REST API to a server database (e.g. Node.js/PHP with MySQL), SMS gateway and e-mail service.

### 6.4 Communication Interfaces

HTTP/HTTPS when hosted. Production deployment shall use HTTPS only.

## 7. Use-Case Model

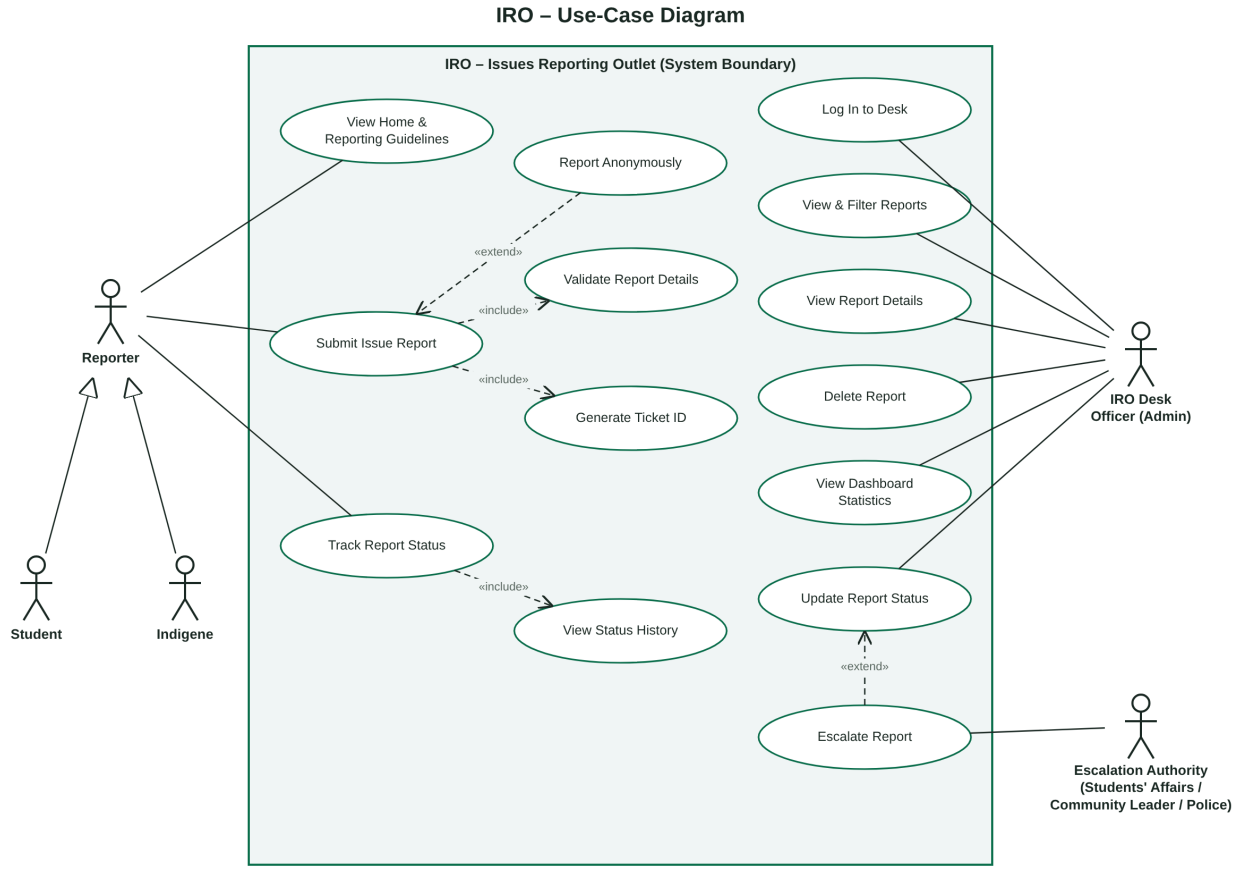



*Figure 1: IRO use-case diagram (source: docs/use-case/IRO_UseCase.drawio)*

### 7.1 Actors

- **Reporter** (generalised from **Student** and **Indigene**) – submits and tracks reports.
- **IRO Desk Officer** – manages reports.
- **Escalation Authority** – receives escalated reports (Students' Affairs, community leaders, police).

### 7.2 Use-Case Descriptions

| Use case | Actor | Main flow | Alternative / notes |
|---|---|---|---|
| UC-01 View Home & Guidelines | Reporter | Reporter opens IRO and reads the process, categories and emergency notice. | – |
| UC-02 Submit Issue Report | Reporter | 1. Choose reporter type. 2. Fill details. 3. Submit. 4. System validates (UC-03), generates ticket (UC-04) and shows it. | Validation fails → message shown, nothing saved. |
| UC-03 Validate Report Details | System («include») | Checks required fields, date, lengths and contact format. | – |
| UC-04 Generate Ticket ID | System («include») | Creates unique IRO-YY-XXXXX ID and first history entry. | – |
| UC-05 Report Anonymously | Reporter («extend» UC-02) | Reporter ticks Anonymous; identity fields are hidden and not stored. | – |

| | | | |
|---|---|---|---|
| UC-06 Track Report Status | Reporter | Enter ticket ID → system shows status and history (UC-07). | Unknown ID → error. |
| UC-07 View Status History | System («include») | Displays every status change with time and note. | – |
| UC-08 Log In to Desk | Desk Officer | Enter passcode → dashboard opens. | Wrong passcode → error. |
| UC-09 View & Filter Reports | Desk Officer | Search by text, filter by status/category/reporter type. | No match → "No reports match". |
| UC-10 View Report Details | Desk Officer | Open a report to see all details and history. | – |
| UC-11 Update Report Status | Desk Officer | Choose new status, add note, save → history updated. | – |
| UC-12 Escalate Report | Desk Officer → Escalation Authority («extend» UC-11) | Set status Escalated with note naming the authority; case is forwarded. | Forwarding is offline in v1.0. |
| UC-13 Delete Report | Desk Officer | Confirm deletion → report removed. | Cancel → nothing changes. |
| UC-14 View Dashboard Statistics | Desk Officer | See totals of all, open, emergency and resolved reports. | – |

## 8. Data Requirements

### 8.1 Issue Object (data dictionary)

| Field | Type | Description / rule |
|---|---|---|
| id | String | Ticket ID, IRO-YY-XXXXX, unique. |
| reporterType | String | Student \| Indigene \| Other (required). |
| anonymous | Boolean | True if reporter chose anonymity. |
| name | String | Reporter's name; empty if anonymous. |
| contact | String | Phone or e-mail; empty if anonymous. |
| matricNo | String | Optional; students only. |
| community | String | Required for indigenes. |
| category | String | One of the configured categories. |
| severity | String | Low \| Medium \| High \| Emergency. |
| location | String | Where the incident happened. |
| incidentDate | Date (YYYY-MM-DD) | Not in the future. |

| | | |
|---|---|---|
| title | String (5–100) | Short summary. |
| description | String (20–2000) | Full account. |
| status | String | Submitted \| Under Review \| Escalated \| Resolved \| Closed. |
| createdAt | ISO date-time | Time of submission. |
| history | Array of {status, note, at} | Append-only log of status changes. |

### 8.2 Configuration Lists

**Categories:** Security / Safety; Accommodation & Rent; Land & Property; Harassment / Assault; Cultural & Festival Conflicts; Sanitation & Environment; Water, Power & Utilities; Transport & Okada; Market & Pricing; Other.
**Communities:** Akungba, Iwaro, Etioro, Ayegunle, Oke-Oka, Other.

## 9. Problem Analysis (Ishikawa) and Requirements Traceability

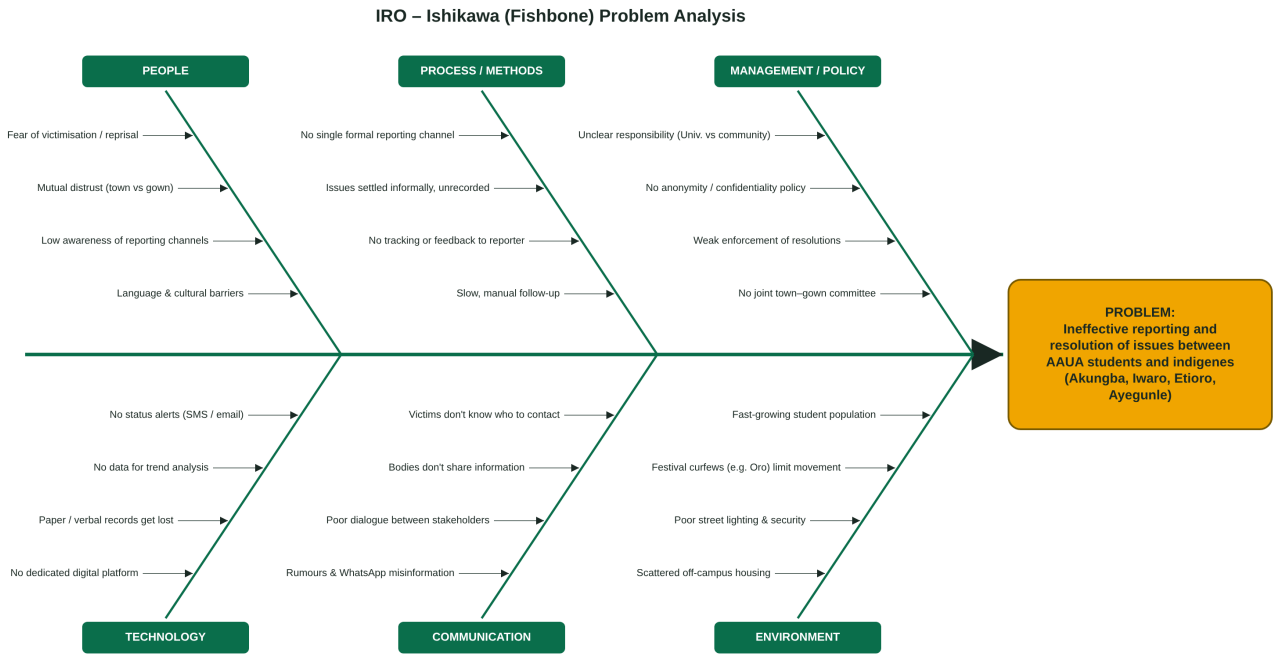



*Figure 2: Ishikawa (fishbone) analysis of the problem (source: docs/ishikawa/IRO_Ishikawa.drawio)*

| Cause category | Root cause | Addressed by |
|---|---|---|
| People | Fear of victimisation / reprisal | FR-05, FR-11, NFR-03 |
| People | Low awareness of reporting channels | FR-01, FR-02 |
| Process / Methods | No single formal reporting channel | FR-03 – FR-09 |
| Process / Methods | No tracking or feedback to reporter | FR-08, FR-10, FR-17 |
| Management / Policy | Unclear responsibility; weak enforcement | FR-18, FR-17 (logged accountability) |

| | | |
|---|---|---|
| Management / Policy | No anonymity / confidentiality policy | FR-05, NFR-03, NFR-07 |
| Technology | No dedicated platform; records get lost | Whole system; NFR-12 |
| Technology | No data for trend analysis | FR-14, FR-15, FR-22 |
| Communication | Victims don't know who to contact; bodies don't share information | FR-01, FR-18 |
| Environment | Scattered housing; insecurity | FR-06 (location, severity), FR-14 (emergencies) |

## 10. Appendices

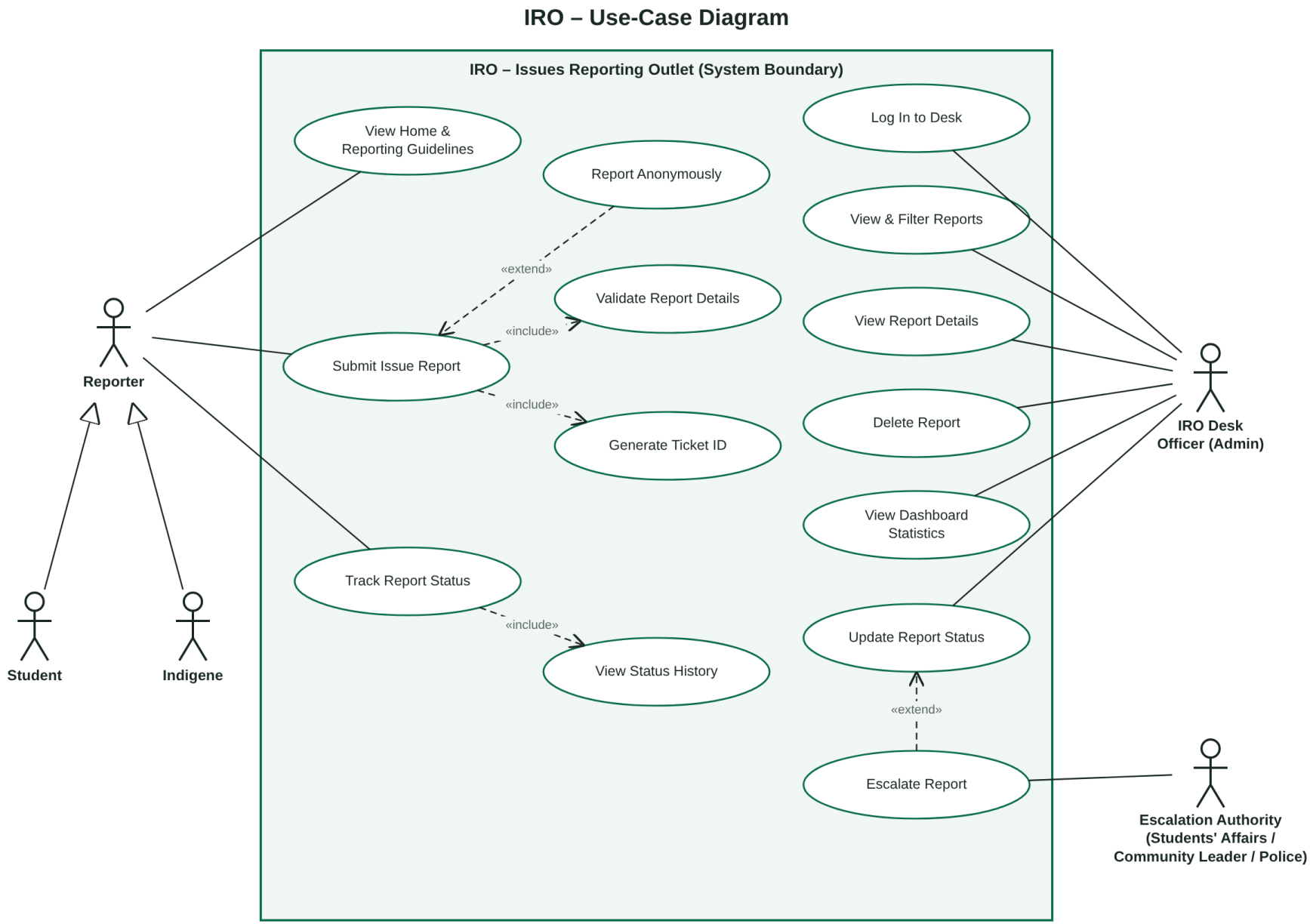
**Appendix A – Questionnaire**

A Google Form questionnaire was designed for both students and indigenes, branching after the first question so that each group answers questions relevant to them (level/faculty/residence for students; community/occupation/years of residence for indigenes) before common sections on issues experienced, current reporting channels and desired IRO features. See `docs/questionnaire/`.

**Appendix B – Future Enhancements**

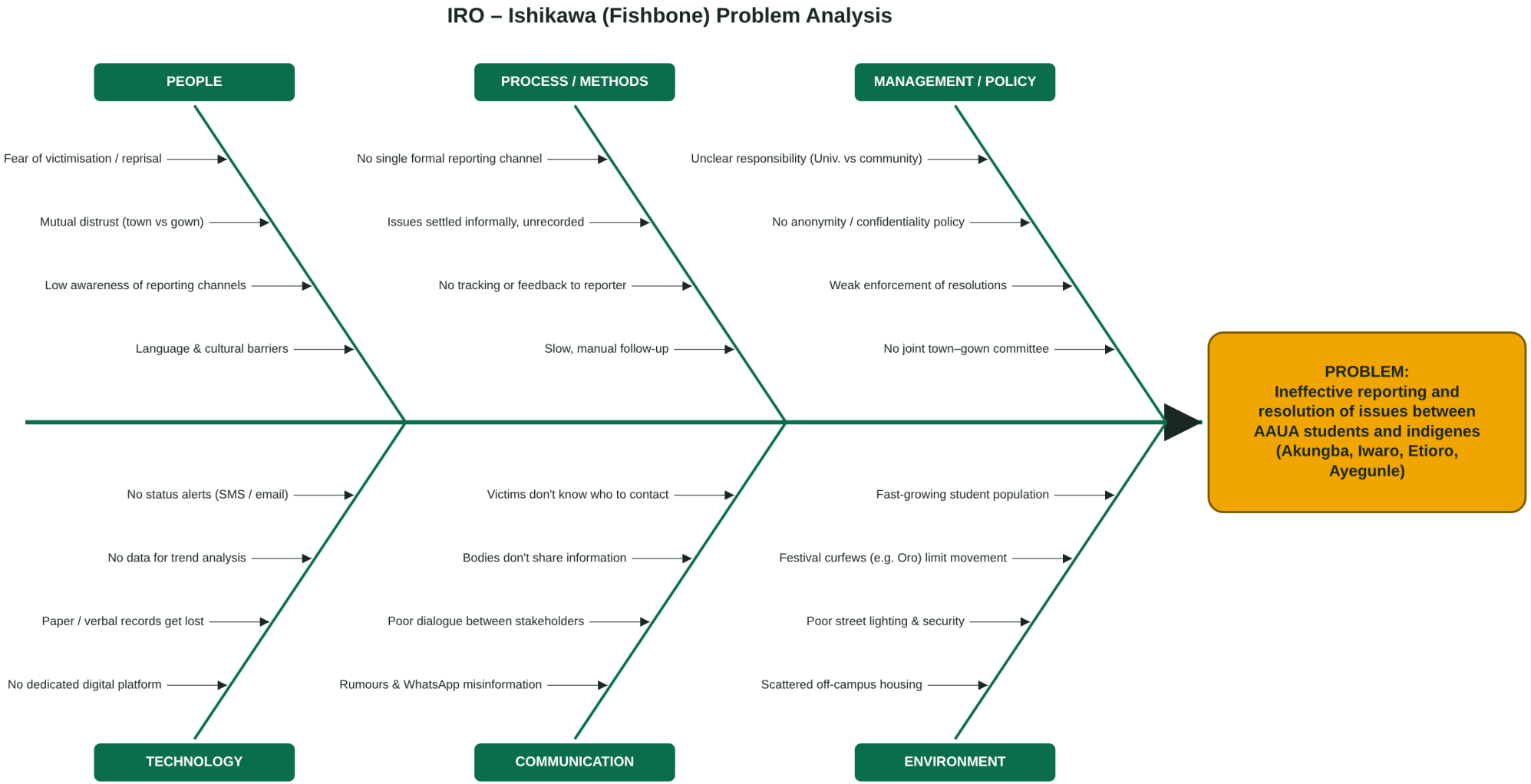
- Server database and REST API; multiple officer accounts with roles.
- SMS / e-mail / WhatsApp status notifications; USSD reporting for feature phones.
- Photo and video evidence upload.
- Yoruba language option.
- Escalation authorities log in to respond directly.
- Monthly public report of resolved issues by category and community.

## 2. IRO Use-Case Diagram




*Source: docs/use-case/IRO_UseCase.drawio*

## 3. IRO Ishikawa (Fishbone) Problem Analysis




*Source: docs/ishikawa/IRO_Ishikawa.drawio*

# IRO – Issues Reporting Outlet: Object Listing

CSC426 Weekly Dev – Weekend 3 · Joshua Omolafe (Joshua989)

IRO is written in plain JavaScript as a small set of singleton objects. Each page has one controller object, and all of them share the configuration object `IRO` and the data-layer object `IssueStore`. The tables below list every object with its properties and methods.

## 1. Object summary

| Object | File | Kind | Used by | Purpose |
|---|---|---|---|---|
| `IRO` | js/store.js | Configuration | All pages | Constant lists and settings. |
| `IssueStore` | js/store.js | Data layer | All controllers | Create, read, update and delete issues; statistics. |
| Helper functions | js/store.js | Utilities | All controllers | Escaping, date formatting, badges, filling selects. |
| `ReportForm` | js/report.js | Controller | report.html | Report submission. |
| `Tracker` | js/track.js | Controller | track.html | Status tracking by ticket ID. |
| `AdminDesk` | js/admin.js | Controller | admin.html | Desk officer dashboard. |
| Issue | (data record) | Entity | IssueStore | One reported issue stored in localStorage. |

## 2. IRO (configuration object)

| Property | Type | Value / description |
|---|---|---|
| `STORAGE_KEY` | String | "iro_issues" – localStorage key holding all issues. |
| `ADMIN_PASSCODE` | String | "iro2026" – demo passcode for the admin desk. |
| `CATEGORIES` | Array<String> | 10 issue categories (Security / Safety … Other). |
| `COMMUNITIES` | Array<String> | Akungba, Iwaro, Etioro, Ayegunle, Oke-Oka, Other. |
| `STATUSES` | Array<String> | Submitted, Under Review, Escalated, Resolved, Closed. |
| `SEVERITIES` | Array<String> | Low, Medium, High, Emergency. |

## 3. IssueStore (data-layer object)

| Method | Returns | Description |
|---|---|---|
| `getAll()` | Array<Issue> | Reads all issues from localStorage, newest first; returns [] if data is missing or corrupt. |
| `saveAll(list)` | void | Writes the whole list back to localStorage as JSON. |
| `generateId()` | String | Builds a unique ticket ID IRO-YY-XXXXX from an unambiguous alphabet. |
| `create(data)` | Issue | Builds an Issue from form data (drops name/contact if anonymous), sets status Submitted with first history entry, saves it. |
| `findById(id)` | Issue \| null | Case-insensitive lookup by ticket ID. |
| `updateStatus(id, status, note)` | Issue \| null | Sets the new status and appends {status, note, at} to history. |
| `remove(id)` | void | Deletes an issue. |

| | | |
|---|---|---|
| `stats()` | Object | { total, open, emergency, byStatus } for the dashboard. |

### 4. Helper functions (global)

| Function | Returns | Description |
|---|---|---|
| `escapeHtml(str)` | String | Replaces & < > " ' with HTML entities to prevent XSS. |
| `formatDate(iso)` | String | Formats an ISO date, e.g. "7 Jul 2026, 14:05". |
| `statusBadge(status)` | String (HTML) | Coloured badge markup for a status. |
| `fillSelect(select, items, placeholder)` | void | Fills a <select> with option elements. |

### 5. ReportForm (controller for report.html)

| Member | Type | Description |
|---|---|---|
| `form` | HTMLFormElement | The #reportForm element. |
| `el(id)` | Function | Shortcut for document.getElementById. |
| `init()` | Method | Fills drop-downs, sets the max date, attaches event listeners. |
| `toggleReporterFields()` | Method | Shows matric field for students, community list for indigenes. |
| `toggleIdentity()` | Method | Hides name/contact when Anonymous is ticked. |
| `collect()` | Method → Object | Reads every form field into a plain object. |
| `validate(d)` | Method → String | Returns an error message, or "" if valid. |
| `submit(e)` | Method | Validates, calls IssueStore.create(), shows the ticket ID. |

### 6. Tracker (controller for track.html)

| Member | Type | Description |
|---|---|---|
| `el(id)` | Function | Shortcut for document.getElementById. |
| `init()` | Method | Attaches the form handler; auto-tracks an `?id=` in the URL. |
| `show(id)` | Method | Finds the issue and renders title, meta, status and history (no personal data). |

### 7. AdminDesk (controller for admin.html)

| Member | Type | Description |
|---|---|---|
| `SESSION_KEY` | String | "iro_admin" – sessionStorage flag for a logged-in officer. |
| `selectedId` | String \| null | Ticket ID of the report open in the detail panel. |
| `el(id)` | Function | Shortcut for document.getElementById. |
| `init()` | Method | Fills filters, attaches listeners, restores an existing session. |
| `login(e)` / `logout()` | Method | Checks passcode and opens the dashboard / ends the session. |
| `showDashboard()` | Method | Hides login card, shows dashboard, renders data. |

| | | |
|---|---|---|
| `filtered()` | Method → Array | Issues matching search text and status/category/reporter filters. |
| `render()` | Method | Updates statistics cards and the report table. |
| `openDetail(id)` | Method | Shows full report details, reporter identity and history. |
| `saveUpdate(e)` | Method | Calls IssueStore.updateStatus() with the chosen status and note. |
| `deleteSelected()` | Method | Confirms, then deletes the open report. |

### 8. Issue (data record)

| Property | Type | Description |
|---|---|---|
| `id` | String | Ticket ID. |
| `reporterType` | String | Student \| Indigene \| Other. |
| `anonymous` | Boolean | Reporter chose anonymity. |
| `name`, `contact` | String | Empty when anonymous. |
| `matricNo`, `community` | String | Student matric number / indigene community. |
| `category`, `severity` | String | From IRO.CATEGORIES / IRO.SEVERITIES. |
| `location`, `incidentDate` | String | Where and when (YYYY-MM-DD). |
| `title`, `description` | String | Summary and full account. |
| `status` | String | Current status. |
| `createdAt` | String (ISO) | Submission time. |
| `history` | Array<{status, note, at}> | Append-only status log. |

### 9. HTML page objects (main DOM elements)

| Page | Key element IDs |
|---|---|
| index.html | #categoryList |
| report.html | #reportForm, #reporterType, #matricGroup, #matricNo, #communityGroup, #community, #anonymous, #identityGroup, #name, #contact, #category, #severity, #location, #incidentDate, #title, #description, #formError, #formCard, #successCard, #ticketId, #trackLink |
| track.html | #trackForm, #ticketInput, #trackError, #resultCard, #resTitle, #resMeta, #resStatus, #resHistory |
| admin.html | #loginCard, #loginForm, #passcode, #loginError, #dashboard, #logoutBtn, #statTotal, #statOpen, #statEmergency, #statResolved, #search, #filterStatus, #filterCategory, #filterReporter, #issueRows, #emptyMsg, #detailCard, #dTitle, #dMeta, #dReporter, #dDesc, #dHistory, #updateForm, #newStatus, #note, #deleteBtn |

# IRO – Issues Reporting Outlet: Code Listing

CSC426 Weekly Dev – Weekend 3 · Joshua Omolafe (Joshua989)

**app/index.html (71 lines)**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>IRO – Issues Reporting Outlet</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <nav class="nav">
      <a class="brand" href="index.html">IR<span>O</span></a>
      <ul>
        <li><a class="link active" href="index.html">Home</a></li>
        <li><a class="link" href="report.html">Report an Issue</a></li>
        <li><a class="link" href="track.html">Track Report</a></li>
        <li><a class="link" href="admin.html">Admin</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section class="hero">
      <h1>Issues Reporting Outlet</h1>
      <p>
        One safe place for AAUA students and the indigenes of Akungba, Iwaro,
        Etioro, Ayegunle and neighbouring communities to report issues that
        affect town–gown relations — and to follow them until they are resolved.
      </p>
      <a class="btn" href="report.html">Report an Issue</a>
      <a class="btn secondary" href="track.html">Track My Report</a>
    </section>

    <section class="grid">
      <div class="card">
        <h3>1. Report</h3>
        <p>Describe what happened, where and when. You can stay anonymous.</p>
      </div>
      <div class="card">
        <h3>2. Get a ticket</h3>
        <p>You receive a ticket ID (e.g. <code>IRO-26-4F7K2</code>). Keep it safe.</p>
      </div>
      <div class="card">
        <h3>3. Track</h3>
        <p>Use your ticket ID to see the status and any updates from the IRO desk.</p>
      </div>
      <div class="card">
        <h3>4. Resolution</h3>
        <p>The IRO desk reviews, escalates to the right authority and records the outcome.</p>
      </div>
    </section>

    <section class="card" style="margin-top:24px">
```

```
      <h2>What can I report?</h2>
      <p id="categoryList"></p>
      <div class="notice">
        <strong>Emergency?</strong> IRO is not an emergency line. If anyone is in
        immediate danger, contact campus security or the nearest police division first,
        then file a report here.
      </div>
    </section>
  </main>

  <footer>IRO &copy; 2026 · CSC426 Weekly Dev · Adekunle Ajasin University, Akungba-Akoko</footer>

  <script src="js/store.js"></script>
  <script>
    document.getElementById("categoryList").textContent = IRO.CATEGORIES.join(" · ");
  </script>
</body>
</html>
```

**app/report.html (107 lines)**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Report an Issue – IRO</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <nav class="nav">
      <a class="brand" href="index.html">IR<span>O</span></a>
      <ul>
        <li><a class="link" href="index.html">Home</a></li>
        <li><a class="link active" href="report.html">Report an Issue</a></li>
        <li><a class="link" href="track.html">Track Report</a></li>
        <li><a class="link" href="admin.html">Admin</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <div class="card" id="formCard">
      <h2>Report an Issue</h2>
      <p>Fields marked <span class="opt">(optional)</span> can be left blank.</p>

      <form id="reportForm" novalidate>
        <h3>About you</h3>
        <div class="row">
          <div>
            <label for="reporterType">I am a</label>
            <select id="reporterType" required>
              <option value="">-- Select --</option>
              <option>Student</option>
              <option>Indigene</option>
              <option>Other</option>
            </select>
          </div>
          <div id="matricGroup" class="hidden">
```

```
            <label for="matricNo">Matric number <span class="opt">(optiona
l)</span></label>
            <input id="matricNo" placeholder="e.g. 210404012">
          </div>
          <div id="communityGroup" class="hidden">
            <label for="community">Community</label>
            <select id="community"></select>
          </div>
        </div>

        <label class="check"><input type="checkbox" id="anonymous"> Report
anonymously (your name and contact will not be stored)</label>

        <div class="row" id="identityGroup">
          <div>
            <label for="name">Full name <span class="opt">(optional)</span>
</label>
            <input id="name" autocomplete="name">
          </div>
          <div>
            <label for="contact">Phone or email <span class="opt">(optiona
l)</span></label>
            <input id="contact" autocomplete="email">
          </div>
        </div>

        <h3>The issue</h3>
        <div class="row">
          <div>
            <label for="category">Category</label>
            <select id="category" required></select>
          </div>
          <div>
            <label for="severity">Severity</label>
            <select id="severity" required></select>
          </div>
        </div>
        <div class="row">
          <div>
            <label for="location">Location</label>
            <input id="location" required placeholder="e.g. Ibaka junction,
Iwaro">
          </div>
          <div>
            <label for="incidentDate">Date of incident</label>
            <input type="date" id="incidentDate" required>
          </div>
        </div>
        <label for="title">Short title</label>
        <input id="title" required maxlength="100" placeholder="e.g. Landlo
rd increased rent mid-session">
        <label for="description">Describe what happened</label>
        <textarea id="description" required maxlength="2000"></textarea>

        <p id="formError" class="error"></p>
        <button class="btn" type="submit">Submit Report</button>
      </form>
    </div>

    <div class="card hidden" id="successCard">
      <h2>Report submitted</h2>
      <p>Thank you. Your report has been received. Save this ticket ID — yo
u need it to track your report:</p>
      <div class="notice"><span class="ticket" id="ticketId"></span></div>
      <a class="btn secondary" id="trackLink" href="track.html">Track this
report</a>
      <a class="btn" href="report.html">Report another issue</a>
```

```
    </div>
  </main>

  <footer>IRO &copy; 2026 · CSC426 Weekly Dev · Adekunle Ajasin University, Akungba-Akoko</footer>

  <script src="js/store.js"></script>
  <script src="js/report.js"></script>
</body>
</html>
```

**app/track.html (46 lines)**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Track Report – IRO</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <nav class="nav">
      <a class="brand" href="index.html">IR<span>O</span></a>
      <ul>
        <li><a class="link" href="index.html">Home</a></li>
        <li><a class="link" href="report.html">Report an Issue</a></li>
        <li><a class="link active" href="track.html">Track Report</a></li>
        <li><a class="link" href="admin.html">Admin</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <div class="card">
      <h2>Track a Report</h2>
      <form id="trackForm" class="toolbar">
        <input id="ticketInput" placeholder="Enter ticket ID, e.g. IRO-26-4F7K2" required>
        <button class="btn" type="submit" style="flex:0 0 auto">Track</button>
      </form>
      <p id="trackError" class="error"></p>
    </div>

    <div class="card hidden" id="resultCard" style="margin-top:16px">
      <h2 id="resTitle"></h2>
      <p id="resMeta"></p>
      <p><strong>Current status:</strong> <span id="resStatus"></span></p>
      <h3>Progress</h3>
      <ul class="timeline" id="resHistory"></ul>
    </div>
  </main>

  <footer>IRO &copy; 2026 · CSC426 Weekly Dev · Adekunle Ajasin University, Akungba-Akoko</footer>

  <script src="js/store.js"></script>
  <script src="js/track.js"></script>
</body>
</html>
```

**app/admin.html (103 lines)**

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Desk – IRO</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <nav class="nav">
      <a class="brand" href="index.html">IR<span>O</span></a>
      <ul>
        <li><a class="link" href="index.html">Home</a></li>
        <li><a class="link" href="report.html">Report an Issue</a></li>
        <li><a class="link" href="track.html">Track Report</a></li>
        <li><a class="link active" href="admin.html">Admin</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <!-- Login -->
    <div class="card" id="loginCard" style="max-width:420px;margin:0 auto">
      <h2>IRO Desk Login</h2>
      <form id="loginForm">
        <label for="passcode">Passcode</label>
        <input type="password" id="passcode" required>
        <p class="error" id="loginError"></p>
        <button class="btn" type="submit">Log in</button>
      </form>
      <p class="opt"><small>Demo passcode: <code>iro2026</code></small></p>
    </div>

    <!-- Dashboard -->
    <div id="dashboard" class="hidden">
      <div style="display:flex;justify-content:space-between;align-items:ce
nter;flex-wrap:wrap;gap:10px">
        <h2 style="margin:0">Issue Management Desk</h2>
        <button class="btn small secondary" id="logoutBtn">Log out</button>
      </div>

      <section class="grid" style="margin:16px 0">
        <div class="card"><div class="stat" id="statTotal">0</div>Total rep
orts</div>
        <div class="card"><div class="stat" id="statOpen">0</div>Open</div>
        <div class="card"><div class="stat" id="statEmergency" style="colo
r:var(--danger)">0</div>Active emergencies</div>
        <div class="card"><div class="stat" id="statResolved">0</div>Resolv
ed</div>
      </section>

      <div class="toolbar">
        <input id="search" placeholder="Search ticket, title or location">
        <select id="filterStatus"></select>
        <select id="filterCategory"></select>
        <select id="filterReporter">
          <option value="">All reporters</option>
          <option>Student</option>
          <option>Indigene</option>
          <option>Other</option>
        </select>
      </div>

      <div class="table-wrap">
        <table>
          <thead>
```

```
            <tr><th>Ticket</th><th>Issue</th><th>Reporter</th><th>Severity</th><th>Status</th><th>Action</th></tr>
          </thead>
          <tbody id="issueRows"></tbody>
        </table>
      </div>
      <p id="emptyMsg" class="hidden">No reports match the current filters.</p>

      <!-- Detail / update panel -->
      <div class="card hidden" id="detailCard" style="margin-top:16px">
        <h3 id="dTitle"></h3>
        <p id="dMeta"></p>
        <p id="dReporter"></p>
        <p id="dDesc" style="white-space:pre-wrap"></p>
        <h4>History</h4>
        <ul class="timeline" id="dHistory"></ul>
        <h4>Update status</h4>
        <form id="updateForm" class="row">
          <div>
            <label for="newStatus">New status</label>
            <select id="newStatus"></select>
          </div>
          <div>
            <label for="note">Note to reporter</label>
            <input id="note" placeholder="e.g. Forwarded to Students' Affairs">
          </div>
          <div style="align-self:end">
            <button class="btn" type="submit">Save update</button>
            <button class="btn small" type="button" id="deleteBtn" style="background:var(--danger);color:#fff">Delete</button>
          </div>
        </form>
      </div>
    </div>
  </main>

  <footer>IRO &copy; 2026 · CSC426 Weekly Dev · Adekunle Ajasin University, Akungba-Akoko</footer>

  <script src="js/store.js"></script>
  <script src="js/admin.js"></script>
</body>
</html>
```

**app/css/style.css (154 lines)**

```
/* IRO – Issues Reporting Outlet : shared stylesheet */
:root {
  --green: #0b6e4f;
  --green-dark: #08513a;
  --gold: #f2a900;
  --bg: #f4f7f5;
  --card: #ffffff;
  --text: #1d2a24;
  --muted: #5c6b64;
  --border: #d6e0db;
  --danger: #b3261e;
  --radius: 10px;
}

* { box-sizing: border-box; }

body {
  margin: 0;
```

```
  font-family: "Segoe UI", Roboto, Arial, sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.5;
}

a { color: var(--green); }

/* ---------- Header / navigation ---------- */
header {
  background: var(--green);
  color: #fff;
}
.nav {
  max-width: 1000px;
  margin: 0 auto;
  padding: 14px 16px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: 10px;
}
.brand { font-size: 1.4rem; font-weight: 700; color: #fff; text-decoration: none; }
.brand span { color: var(--gold); }
.nav ul { list-style: none; margin: 0; padding: 0; display: flex; gap: 6px; flex-wrap: wrap; }
.nav a.link {
  color: #fff;
  text-decoration: none;
  padding: 6px 12px;
  border-radius: 6px;
}
.nav a.link:hover, .nav a.link.active { background: var(--green-dark); }

/* ---------- Layout ---------- */
main { max-width: 1000px; margin: 0 auto; padding: 24px 16px 48px; }

.hero {
  background: linear-gradient(135deg, var(--green), var(--green-dark));
  color: #fff;
  border-radius: var(--radius);
  padding: 36px 28px;
  margin-bottom: 24px;
}
.hero h1 { margin: 0 0 8px; font-size: 1.9rem; }
.hero p { margin: 0 0 18px; max-width: 640px; opacity: .95; }

.grid { display: grid; gap: 16px; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }

.card {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 20px;
}
.card h2, .card h3 { margin-top: 0; color: var(--green-dark); }

/* ---------- Forms ---------- */
form h3 { margin: 24px 0 0; }
form .row { display: grid; gap: 14px; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); }
label { display: block; font-weight: 600; margin: 12px 0 4px; }
.opt { font-weight: 400; color: var(--muted); font-size: .85rem; }
input, select, textarea {
```

```
  width: 100%;
  padding: 10px;
  border: 1px solid var(--border);
  border-radius: 6px;
  font: inherit;
  background: #fff;
}
textarea { min-height: 120px; resize: vertical; }
input:focus, select:focus, textarea:focus { outline: 2px solid var(--gold); border-color: var(--gold); }
.check { display: flex; align-items: center; gap: 8px; font-weight: 400; }
.check input { width: auto; }
.hidden { display: none !important; }

.btn {
  display: inline-block;
  background: var(--gold);
  color: #1d2a24;
  border: none;
  padding: 10px 20px;
  border-radius: 6px;
  font-weight: 700;
  cursor: pointer;
  text-decoration: none;
  font-size: 1rem;
}
.btn:hover { filter: brightness(.95); }
.btn.secondary { background: var(--green); color: #fff; }
.btn.small { padding: 5px 10px; font-size: .85rem; }

.error { color: var(--danger); font-size: .9rem; margin-top: 4px; }

.notice {
  background: #e7f4ee;
  border-left: 4px solid var(--green);
  padding: 14px 16px;
  border-radius: 6px;
  margin: 16px 0;
}
.ticket { font-family: monospace; font-size: 1.4rem; font-weight: 700; letter-spacing: 1px; }

/* ---------- Status badges & timeline ---------- */
.badge {
  display: inline-block;
  padding: 2px 10px;
  border-radius: 999px;
  font-size: .8rem;
  font-weight: 700;
  white-space: nowrap;
}
.badge.Submitted   { background: #e3e8ff; color: #2b3a99; }
.badge.Under-Review{ background: #fff3cd; color: #7a5a00; }
.badge.Escalated   { background: #fde2e1; color: #8c1d18; }
.badge.Resolved    { background: #d9f2e4; color: #0b6e4f; }
.badge.Closed      { background: #e4e4e4; color: #444; }

.timeline { list-style: none; padding: 0; margin: 0; border-left: 3px solid var(--border); }
.timeline li { padding: 0 0 14px 16px; position: relative; }
.timeline li::before {
  content: ""; position: absolute; left: -8px; top: 5px;
  width: 13px; height: 13px; border-radius: 50%; background: var(--green);
}
.timeline small { color: var(--muted); display: block; }

/* ---------- Admin table ---------- */
```

```
.toolbar { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 12px; }
.toolbar > * { flex: 1 1 160px; }
.table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; background: #fff; }
th, td { padding: 10px; border-bottom: 1px solid var(--border); text-align: left; vertical-align: top; font-size: .92rem; }
th { background: #eef3f0; }
.stat { font-size: 1.8rem; font-weight: 700; color: var(--green); }

footer { text-align: center; color: var(--muted); font-size: .85rem; padding: 20px 16px; }
```

**app/js/store.js (160 lines)**

```
/*
 * IRO – Issues Reporting Outlet
 * store.js : data layer (IssueStore object).
 *
 * The mini version keeps all reports in the browser's localStorage so the
 * app runs with no server. Every other script talks to data ONLY through
 * IssueStore, so swapping localStorage for a real backend API later only
 * means rewriting this file.
 */

const IRO = {
  STORAGE_KEY: "iro_issues",
  ADMIN_PASSCODE: "iro2026", // demo only – a real deployment authenticates on a server

  CATEGORIES: [
    "Security / Safety",
    "Accommodation & Rent",
    "Land & Property",
    "Harassment / Assault",
    "Cultural & Festival Conflicts",
    "Sanitation & Environment",
    "Water, Power & Utilities",
    "Transport & Okada",
    "Market & Pricing",
    "Other"
  ],

  COMMUNITIES: ["Akungba", "Iwaro", "Etioro", "Ayegunle", "Oke-Oka", "Other"],

  STATUSES: ["Submitted", "Under Review", "Escalated", "Resolved", "Closed"],

  SEVERITIES: ["Low", "Medium", "High", "Emergency"]
};

const IssueStore = {
  /** Return every saved issue (newest first). */
  getAll() {
    try {
      const raw = localStorage.getItem(IRO.STORAGE_KEY);
      const list = raw ? JSON.parse(raw) : [];
      return list.sort((a, b) => b.createdAt.localeCompare(a.createdAt));
    } catch (e) {
      return [];
    }
  },

  /** Persist the full list. */
  saveAll(list) {
```

```
    localStorage.setItem(IRO.STORAGE_KEY, JSON.stringify(list));
  },

  /** Create a unique, human-friendly ticket ID such as IRO-26-4F7K2. */
  generateId() {
    const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"; // no 0/O/1/I to avoid confusion
    const year = String(new Date().getFullYear()).slice(-2);
    let id;
    do {
      let code = "";
      for (let i = 0; i < 5; i++) code += chars[Math.floor(Math.random() * chars.length)];
      id = `IRO-${year}-${code}`;
    } while (this.findById(id));
    return id;
  },

  /** Add a new issue and return it. */
  create(data) {
    const now = new Date().toISOString();
    const issue = {
      id: this.generateId(),
      reporterType: data.reporterType,          // "Student" | "Indigene" | "Other"
      anonymous: !!data.anonymous,
      name: data.anonymous ? "" : data.name.trim(),
      contact: data.anonymous ? "" : data.contact.trim(),
      matricNo: data.matricNo ? data.matricNo.trim() : "",
      community: data.community || "",
      category: data.category,
      severity: data.severity,
      location: data.location.trim(),
      incidentDate: data.incidentDate,
      title: data.title.trim(),
      description: data.description.trim(),
      status: "Submitted",
      createdAt: now,
      history: [{ status: "Submitted", note: "Report received by IRO.", at: now }]
    };
    const list = this.getAll();
    list.push(issue);
    this.saveAll(list);
    return issue;
  },

  /** Find one issue by its ticket ID (case-insensitive). */
  findById(id) {
    if (!id) return null;
    const key = id.trim().toUpperCase();
    return this.getAll().find(i => i.id === key) || null;
  },

  /** Change an issue's status and log the change in its history. */
  updateStatus(id, status, note) {
    const list = this.getAll();
    const issue = list.find(i => i.id === id);
    if (!issue) return null;
    issue.status = status;
    issue.history.push({ status, note: note || "", at: new Date().toISOString() });
    this.saveAll(list);
    return issue;
  },

  /** Remove an issue permanently (admin only). */
```

```
  remove(id) {
    this.saveAll(this.getAll().filter(i => i.id !== id));
  },

  /** Counts used by the admin dashboard. */
  stats() {
    const list = this.getAll();
    const byStatus = {};
    IRO.STATUSES.forEach(s => (byStatus[s] = 0));
    list.forEach(i => byStatus[i.status]++);
    return {
      total: list.length,
      open: list.length - byStatus["Resolved"] - byStatus["Closed"],
      emergency: list.filter(i => i.severity === "Emergency" && i.status !
= "Closed").length,
      byStatus
    };
  }
};

/* ---------- Small helpers shared by every page ---------- */

/** Escape text before inserting it into HTML (prevents script injection).
*/
function escapeHtml(str) {
  return String(str).replace(/[&<>"']/g, c => ({
    "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;"
  })[c]);
}

/** Format an ISO date for display, e.g. 7 Jul 2026, 14:05. */
function formatDate(iso) {
  return new Date(iso).toLocaleString("en-GB", {
    day: "numeric", month: "short", year: "numeric", hour: "2-digit", minut
e: "2-digit"
  });
}

/** Status badge markup; CSS class uses a hyphen in place of spaces. */
function statusBadge(status) {
  return `<span class="badge ${status.replace(/\s/g, "-")}">${escapeHtml(st
atus)}</span>`;
}

/** Fill a <select> element with a list of options. */
function fillSelect(select, items, placeholder) {
  select.innerHTML = placeholder ? `<option value="">${placeholder}</option
>` : "";
  items.forEach(item => {
    const opt = document.createElement("option");
    opt.value = item;
    opt.textContent = item;
    select.appendChild(opt);
  });
}
```

**app/js/report.js (81 lines)**

```
/*
 * IRO – report.js : ReportForm object (controls report.html).
 */

const ReportForm = {
  form: document.getElementById("reportForm"),
  el: id => document.getElementById(id),

```

```
  init() {
    fillSelect(this.el("category"), IRO.CATEGORIES, "-- Select category --");
    fillSelect(this.el("severity"), IRO.SEVERITIES, "-- Select severity --");
    fillSelect(this.el("community"), IRO.COMMUNITIES, "-- Select community --");
    this.el("incidentDate").max = new Date().toISOString().slice(0, 10);

    this.el("reporterType").addEventListener("change", () => this.toggleReporterFields());
    this.el("anonymous").addEventListener("change", () => this.toggleIdentity());
    this.form.addEventListener("submit", e => this.submit(e));
  },

  /** Students may give a matric number; indigenes pick their community. */
  toggleReporterFields() {
    const type = this.el("reporterType").value;
    this.el("matricGroup").classList.toggle("hidden", type !== "Student");
    this.el("communityGroup").classList.toggle("hidden", type !== "Indigene");
  },

  toggleIdentity() {
    this.el("identityGroup").classList.toggle("hidden", this.el("anonymous").checked);
  },

  /** Read all fields into a plain object. */
  collect() {
    return {
      reporterType: this.el("reporterType").value,
      matricNo: this.el("matricNo").value,
      community: this.el("community").value,
      anonymous: this.el("anonymous").checked,
      name: this.el("name").value,
      contact: this.el("contact").value,
      category: this.el("category").value,
      severity: this.el("severity").value,
      location: this.el("location").value,
      incidentDate: this.el("incidentDate").value,
      title: this.el("title").value,
      description: this.el("description").value
    };
  },

  /** Return an error message, or "" when the data is valid. */
  validate(d) {
    if (!d.reporterType) return "Please tell us whether you are a student or an indigene.";
    if (d.reporterType === "Indigene" && !d.community) return "Please select your community.";
    if (!d.category) return "Please choose a category.";
    if (!d.severity) return "Please choose a severity.";
    if (!d.location.trim()) return "Please enter where the issue happened.";
    if (!d.incidentDate) return "Please enter the date of the incident.";
    if (d.incidentDate > new Date().toISOString().slice(0, 10)) return "The incident date cannot be in the future.";
    if (d.title.trim().length < 5) return "Please give a short title (at least 5 characters).";
    if (d.description.trim().length < 20) return "Please describe the issue in at least 20 characters.";
    if (!d.anonymous && d.contact.trim() && !/^(\+?\d{10,14}|[^@\s]+@[^@\s]+\.[^@\s]+)$/.test(d.contact.trim()))
      return "Contact must be a valid phone number or email address.";
```

```
    return "";
  },

  submit(e) {
    e.preventDefault();
    const data = this.collect();
    const error = this.validate(data);
    this.el("formError").textContent = error;
    if (error) return;

    const issue = IssueStore.create(data);
    this.el("ticketId").textContent = issue.id;
    this.el("trackLink").href = "track.html?id=" + encodeURIComponent(issue.id);
    this.el("formCard").classList.add("hidden");
    this.el("successCard").classList.remove("hidden");
    window.scrollTo(0, 0);
  }
};

ReportForm.init();
```

**app/js/track.js (42 lines)**

```
/*
 * IRO – track.js : Tracker object (controls track.html).
 * Shows only non-identifying details so a leaked ticket ID reveals nothing personal.
 */

const Tracker = {
  el: id => document.getElementById(id),

  init() {
    this.el("trackForm").addEventListener("submit", e => {
      e.preventDefault();
      this.show(this.el("ticketInput").value);
    });

    // Allow links such as track.html?id=IRO-26-4F7K2
    const id = new URLSearchParams(location.search).get("id");
    if (id) {
      this.el("ticketInput").value = id;
      this.show(id);
    }
  },

  show(id) {
    const issue = IssueStore.findById(id);
    this.el("resultCard").classList.toggle("hidden", !issue);
    this.el("trackError").textContent = issue ? "" : "No report found with that ticket ID. Check it and try again.";
    if (!issue) return;

    this.el("resTitle").textContent = issue.title;
    this.el("resMeta").textContent =
      `${issue.id} · ${issue.category} · ${issue.severity} severity · reported ${formatDate(issue.createdAt)}`;
    this.el("resStatus").innerHTML = statusBadge(issue.status);
    this.el("resHistory").innerHTML = issue.history.map(h => `
      <li>
        ${statusBadge(h.status)}
        <small>${formatDate(h.at)}</small>
        ${h.note ? `<div>${escapeHtml(h.note)}</div>` : ""}
      </li>`).join("");
  }
```

```
};

Tracker.init();
```

**app/js/admin.js (122 lines)**

```
/*
 * IRO – admin.js : AdminDesk object (controls admin.html).
 * Lets the IRO desk officer view, filter and update reports.
 */

const AdminDesk = {
  SESSION_KEY: "iro_admin",
  selectedId: null,
  el: id => document.getElementById(id),

  init() {
    fillSelect(this.el("filterStatus"), IRO.STATUSES, "All statuses");
    fillSelect(this.el("filterCategory"), IRO.CATEGORIES, "All categorie
s");
    fillSelect(this.el("newStatus"), IRO.STATUSES);

    this.el("loginForm").addEventListener("submit", e => this.login(e));
    this.el("logoutBtn").addEventListener("click", () => this.logout());
    ["search", "filterStatus", "filterCategory", "filterReporter"].forEach
(id =>
      this.el(id).addEventListener("input", () => this.render()));
    this.el("issueRows").addEventListener("click", e => {
      const btn = e.target.closest("button[data-id]");
      if (btn) this.openDetail(btn.dataset.id);
    });
    this.el("updateForm").addEventListener("submit", e => this.saveUpdate
(e));
    this.el("deleteBtn").addEventListener("click", () => this.deleteSelecte
d());

    if (sessionStorage.getItem(this.SESSION_KEY) === "1") this.showDashboar
d();
  },

  login(e) {
    e.preventDefault();
    if (this.el("passcode").value === IRO.ADMIN_PASSCODE) {
      sessionStorage.setItem(this.SESSION_KEY, "1");
      this.showDashboard();
    } else {
      this.el("loginError").textContent = "Incorrect passcode.";
    }
  },

  logout() {
    sessionStorage.removeItem(this.SESSION_KEY);
    location.reload();
  },

  showDashboard() {
    this.el("loginCard").classList.add("hidden");
    this.el("dashboard").classList.remove("hidden");
    this.render();
  },

  /** Issues that match the search box and the three filters. */
  filtered() {
    const q = this.el("search").value.trim().toLowerCase();
    const st = this.el("filterStatus").value;
    const cat = this.el("filterCategory").value;
```

```
    const rep = this.el("filterReporter").value;
    return IssueStore.getAll().filter(i =>
      (!st || i.status === st) &&
      (!cat || i.category === cat) &&
      (!rep || i.reporterType === rep) &&
      (!q || [i.id, i.title, i.location].some(f => f.toLowerCase().includes(q))));
  },

  render() {
    const s = IssueStore.stats();
    this.el("statTotal").textContent = s.total;
    this.el("statOpen").textContent = s.open;
    this.el("statEmergency").textContent = s.emergency;
    this.el("statResolved").textContent = s.byStatus["Resolved"];

    const rows = this.filtered();
    this.el("emptyMsg").classList.toggle("hidden", rows.length > 0);
    this.el("issueRows").innerHTML = rows.map(i => `
      <tr>
        <td><code>${i.id}</code><br><small>${formatDate(i.createdAt)}</small></td>
        <td><strong>${escapeHtml(i.title)}</strong><br><small>${escapeHtml(i.category)} · ${escapeHtml(i.location)}</small></td>
        <td>${i.reporterType}${i.community ? " (" + escapeHtml(i.community) + ")" : ""}</td>
        <td>${i.severity}</td>
        <td>${statusBadge(i.status)}</td>
        <td><button class="btn small secondary" data-id="${i.id}">Open</button></td>
      </tr>`).join("");
  },

  openDetail(id) {
    const i = IssueStore.findById(id);
    if (!i) return;
    this.selectedId = id;
    this.el("dTitle").textContent = `${i.id} – ${i.title}`;
    this.el("dMeta").textContent =
      `${i.category} · ${i.severity} · ${i.location} · incident on ${i.incidentDate}`;
    this.el("dReporter").textContent = i.anonymous
      ? `Reporter: Anonymous ${i.reporterType}`
      : `Reporter: ${i.name || "(no name)"} · ${i.contact || "(no contact)"} · ${i.reporterType}` +
        (i.matricNo ? ` · Matric ${i.matricNo}` : "") + (i.community ? ` · ${i.community}` : "");
    this.el("dDesc").textContent = i.description;
    this.el("dHistory").innerHTML = i.history.map(h =>
      `<li>${statusBadge(h.status)} <small>${formatDate(h.at)}</small>${h.note ? `<div>${escapeHtml(h.note)}</div>` : ""}</li>`
    ).join("");
    this.el("newStatus").value = i.status;
    this.el("note").value = "";
    this.el("detailCard").classList.remove("hidden");
    this.el("detailCard").scrollIntoView({ behavior: "smooth" });
  },

  saveUpdate(e) {
    e.preventDefault();
    if (!this.selectedId) return;
    IssueStore.updateStatus(this.selectedId, this.el("newStatus").value, this.el("note").value.trim());
    this.render();
    this.openDetail(this.selectedId);
  },

```

```
  deleteSelected() {
    if (!this.selectedId || !confirm("Delete this report permanently?")) re
turn;
    IssueStore.remove(this.selectedId);
    this.selectedId = null;
    this.el("detailCard").classList.add("hidden");
    this.render();
  }
};

AdminDesk.init();
```